SpamPatrol

Site web pour le démarchage téléphonique
Techno : React
Outil utilisé : préfixes ARCEP

Idée Clé :

1. Moteur de Recherche des Numéros

- Les utilisateurs saisissent un numéro et obtiennent :
  - Le type de signalement (démarchage, arnaque, sondage).
  - La fréquence des signalements.
  - Des commentaires des utilisateurs sur leur expérience.

2. Signalement Simplifié

- Permettre aux visiteurs de signaler facilement un numéro en quelques clics :
  - Ajouter une catégorie (spam publicitaire, fraude, etc.).
  - Décrire brièvement leur expérience.
  - Mettre en place un système de validation communautaire (votes sur la pertinence des signalements).

3. Dashboard Utilisateur

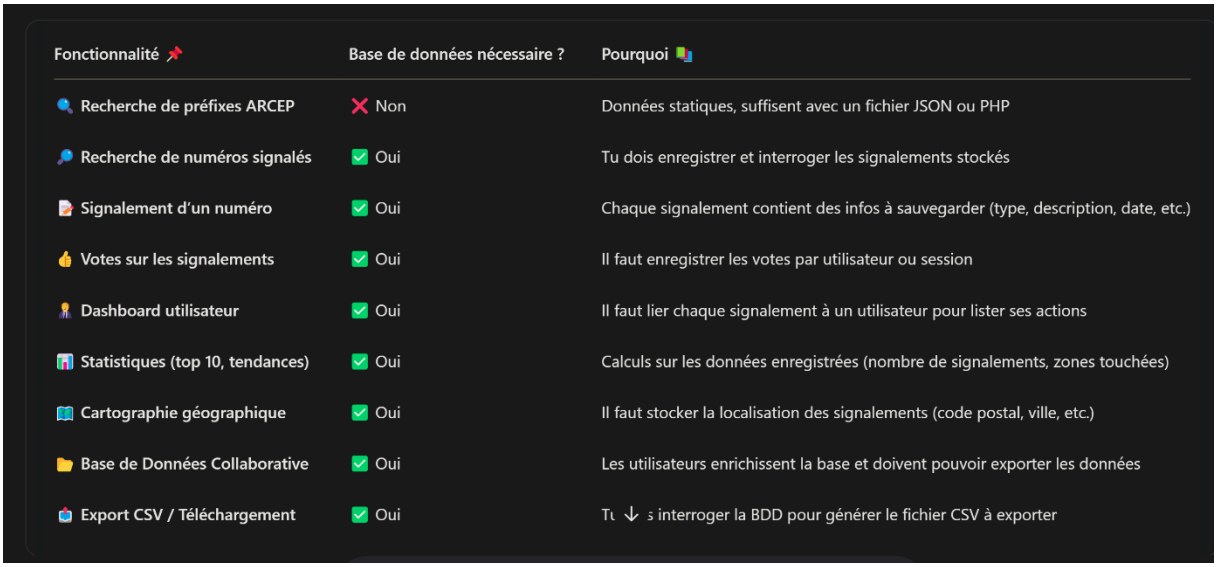
- Offrir un espace personnel où les utilisateurs :
  - Peuvent consulter les numéros qu'ils ont signalés.
  - Gérer une liste noire personnalisée.

5. Statistiques en Temps Réel

- Afficher des tendances :
  - Top 10 des numéros les plus signalés.
  - Cartographie des zones géographiques les plus touchées (par exemple, si les signalements viennent principalement de Paris ou Lyon).

6. Base de Données Collaborative

- Encourager les utilisateurs à enrichir la base avec leurs signalements.
- Fournir un outil pour télécharger les données (par exemple, export CSV des numéros signalés).



Fonctionnalité 📌	Base de données nécessaire ?	Pourquoi 🤔
🔍 Recherche de préfixes ARCEP	❌ Non	Données statiques, suffisent avec un fichier JSON ou PHP
🔎 Recherche de numéros signalés	✅ Oui	Tu dois enregistrer et interroger les signalements stockés
📝 Signalement d'un numéro	✅ Oui	Chaque signalement contient des infos à sauvegarder (type, description, date, etc.)
👍 Votes sur les signalements	✅ Oui	Il faut enregistrer les votes par utilisateur ou session
🧍 Dashboard utilisateur	✅ Oui	Il faut lier chaque signalement à un utilisateur pour lister ses actions
📊 Statistiques (top 10, tendances)	✅ Oui	Calculs sur les données enregistrées (nombre de signalements, zones touchées)
🗺️ Cartographie géographique	✅ Oui	Il faut stocker la localisation des signalements (code postal, ville, etc.)
📂 Base de Données Collaborative	✅ Oui	Les utilisateurs enrichissent la base et doivent pouvoir exporter les données
📥 Export CSV / Téléchargement	✅ Oui	Tu ↓ s interroger la BDD pour générer le fichier CSV à exporter

🧩 Étape 1 – Version sans base de données (fonctionnelle & rapide à coder)

🔹 1. Analyse des préfixes ARCEP

- Stocker les préfixes dans un fichier JSON ou tableau PHP
- Créer un service Symfony PrefixService
- **API** : /api/prefix/{numero} → retourne le type (Mobile, Fixe, etc.)
- Utiliser un code couleur ou une icône selon le type de ligne

🔹 2. Interface utilisateur simple

- Vue.js + Bootstrap (template responsive)
- Champ de recherche pour saisir un numéro
- Affichage dynamique du type de ligne
- Affichage statique de signalements fictifs via fichier JSON :
  - Type de démarchage
  - Catégorie (pub, arnaque, sondage…)
  - Nombre de signalements
  - Commentaires

🔹 3. Intégration IA (facultatif mais stylé)

- API IA type OpenAI pour :
  - Résumer un commentaire
  - Catégoriser automatiquement un signalement
  - Évaluer un score de suspicion
- **API** : /api/analyse/{numero} → retourne résumé + catégorie + score
- Affichage sous forme de badge / carte

🔹 4. Répliques anti-démarchage (fun et viral)

- Fichier JSON ou tableau PHP de répliques drôles
- Classées par catégorie (arnaque, pub, sondage…)
- Affichage aléatoire sur la page d'accueil

🔹 5. Export CSV (à partir du JSON)

- Générer un fichier CSV depuis les signalements fictifs
- Bouton "Exporter" dans l'interface utilisateur

✅ **Cette version est 100% sans base de données**
→ interactive, IA-ready, fun, et déjà très présentable pour un POC 🚀

🔁 Étape 2 – Extension avec base de données (collaboratif et évolutif)

🔹 1. Base de données (Doctrine + migrations)

Entités recommandées :

- Signalement : id, numéro, type, catégorie, commentaire, date, plage de numéros détectée
- Categorie : id, nom, couleur
- Replique : id, contenu, catégorie
- User (facultatif) : id, email, mot de passe (pour dashboard)
- Vote (facultatif) : id, signalement_id, type (up/down), user ou ip
- Localisation (optionnel) : ville, code postal, lat/lon

🔹 2. Fonctionnalités activées avec la BDD

✅ Formulaire de signalement :

- Numéro + catégorie + commentaire
- Appel de l'IA pour enrichir le signalement

- Enregistrement en base

✅ Affichage des signalements (filtrage dynamique) :
- Par numéro
- Par catégorie
- Par popularité

✅ Statistiques :
- Top 10 des numéros les plus signalés
- Évolution par jour / heure
- Carte géographique (zones les plus touchées)
- Détection automatique des **plages de numéros** récurrents (analyse backend)

✅ Dashboard utilisateur :
- Historique de signalements
- Liste noire personnalisée
- Votes sur la pertinence des signalements

✅ Export CSV depuis la base
- Numéros signalés filtrés par catégorie/date

✅ API publique REST :
- Endpoint sécurisé pour développeurs
- Recherche d'un numéro
- Liste des plus signalés
- Envoi de signalements

🧠 **Résumé clair & synthétique**

Fonctionnalité	BDD requise	Description
Analyse préfixe ARCEP	❌	Type de ligne via préfix
Affichage Vue + Bootstrap	❌	Interface responsive & dynamique
IA (résumé + suspicion)	❌	Aide à l'analyse sans stockage
Répliques anti-démarchage	❌	Fun et attractif, classé par catégorie
Signalements fictifs en JSON	❌	Simuler avant BDD
Export CSV JSON	❌	Export manuel des signalements
Signalements réels	✅	Collecte communautaire
Statistiques (Top, temps, zones)	✅	Analyse à partir des données
Détection de plages récurrentes	✅	Clustering de numéros proches
Filtrage par catégorie/popularité	✅	Requêtes dynamiques
Dashboard utilisateur	✅	Espace perso (historique, liste noire)
API publique	✅	Pour usage externe / devs

Étape du projet	Type de composant	Type de test	Outils	Objectif du test
Analyse préfixe ARCEP	PrefixService (Symfony)	Test unitaire	PHPUnit	Vérifier le type de ligne selon le préfixe
API /api/prefix/{numero}	Contrôleur API (Symfony)	Test fonctionnel	PHPUnit + Symfony WebTestCase	Vérifier la réponse API attendue
API /api/analyse/{numero}	Contrôleur IA (Symfony)	Test fonctionnel (avec mock IA)	PHPUnit + Mock	Tester le comportement IA sans appel réel
Recherche numéro	Composant Vue (frontend)	Test unitaire	Vitest ou Jest	Tester le rendu selon l'input utilisateur
Signalements JSON (statique)	Composant Vue	Test unitaire	Vitest ou Jest	Vérifier l'affichage correct des données fictives
Signalement en BDD	Formulaire + Controller	Test fonctionnel	PHPUnit + Symfony	Tester enregistrement et validation
Dashboard utilisateur	Vue + Symfony	Test unitaire & fonctionnel	PHPUnit + Vue test utils	Vérifier affichage de l'historique
Statistiques temps réel	Backend + Graph Vue	Test fonctionnel + UI test	PHPUnit + Vue	Tester les calculs + rendu
Export CSV	Backend export	Test fonctionnel	PHPUnit	Vérifier le format et le contenu du CSV
API publique REST	Endpoints Symfony	Test fonctionnel + sécurité	PHPUnit + Auth mock	Tester accès + données retournées
Répliques aléatoires	Vue Component	Test unitaire	Vitest	Vérifier affichage aléatoire selon la catégorie
Détection plages de numéros	Service Symfony	Test unitaire	PHPUnit	Tester le regroupement logique des numéros proches

Étape	Description	Détails	Temps estimé
1. Configuration de Symfony pour le projet Vue.js	Installer et configurer Symfony pour travailler avec Vue.js.	- Créer un projet Symfony avec composer create-project symfony/skeleton nom-du-projet. - Ajouter le composant Webpack avec composer require symfony/webpack-encore-bundle. - Installer Node.js et npm. - Configurer webpack.config.js pour gérer Vue.js et Bootstrap.	1-2 heures
2. Installation de StarAdmin-Free-Vue-Admin-Template	Ajouter le template Vue.js à ton projet Symfony.	- Télécharger le template StarAdmin-Free-Vue-Admin-Template depuis BootstrapDash. - Extraire les fichiers dans le répertoire assets du projet Symfony. - Installer les dépendances via npm : npm install. - Modifier le fichier webpack.config.js pour inclure les fichiers Vue.js.	1-2 heures
3. Configurer Webpack et Vue.js dans Symfony	Configurer Symfony pour compiler et servir le template Vue.js.	- Mettre à jour webpack.config.js pour gérer Vue.js et inclure les ressources Bootstrap. - Ajouter un fichier app.js dans assets pour importer Vue et démarrer l'application. - Utiliser @vite ou @webpack dans Twig pour charger les ressources (CSS, JS).	1 heure
4. Intégrer Vue.js avec Symfony	Créer des composants Vue.js dynamiques dans ton projet.	- Créer des composants Vue.js dans le dossier assets/vue. - Lier ces composants dans les templates Twig. - Utiliser Vue Router pour gérer les pages (par exemple : page de recherche, page des signalements).	2-3 heures
5. Connexion Backend Symfony et Frontend Vue.js	Configurer les API Symfony pour communiquer avec Vue.js.	- Créer des routes API dans Symfony avec apiPlatform ou FOSRestBundle. - Créer des contrôleurs qui renverront les données JSON	2-3 heures

Étape	Description	Détails	Temps estimé
		pour Vue.js. - Mettre en place des appels AJAX ou des services Vue pour interagir avec les APIs Symfony. - Créer des formulaires Vue.js pour signaler un numéro.	
6. Ajouter des fonctionnalités de recherche et signalement	Mettre en place les fonctionnalités principales du projet (moteur de recherche et signalement).	- Ajouter des champs pour sélectionner la catégorie de signalement et décrire l'expérience. - Enregistrer les données via des appels API Symfony. - Tester la communication entre Symfony et Vue.js.	2-3 heures
7. Tester et déployer le projet	Effectuer des tests pour vérifier l'intégration et le fonctionnement.	- Tester la création de signalements et l’affichage des résultats. - Vérifier l’intégration de BootstrapDash et la réactivité du template. - Déployer le projet sur un serveur local ou distant.	2-3 heures